

SRAI

SETS, LOGIC, RELATIONS AND FUNCTIONS

EXERCISES AND COMPLETE SOLUTIONS

Book 1 · Lesson 2 · PU-B01-C02

Controlled companion aligned with the audited Chapter 2 manuscript and the Colab-compatible M1_N02 notebook

SRAI PRINCIPLE NO. 2 A transparent intelligent system begins with an explicit universe, explicit relations, and testable logical rules.

Owner and author	Mbaye Kebe
Production unit	PU-B01-C02
Companion notebook	M1_N02_sets_logic_relations_functions.ipynb
Edition	Exercises and Solutions v1.0
Release status	Prepared for controlled PU-B01-C02 v1.0.1 release

How to use this companion

This resource separates practice from feedback. Complete Part I without consulting Part II. For computational work, use a fresh runtime and record the environment, inputs, outputs and assertions needed to reproduce your result.

Learning outcomes

- Perform and interpret finite-set operations relative to an explicitly declared universe.
- Construct truth tables and verify logical equivalence.
- Distinguish implication from causation and identify invalid inference patterns.
- Represent relations as ordered pairs and matrices, and classify functions precisely.
- Translate policy language into versioned, testable predicates with explicit missing-data rules.
- Explain decisions produced by a rule system and preserve a reproducible audit trail.

Assessment structure

Part	Focus	Points
A	Conceptual foundations	20
B	Mathematical reasoning	35
C	Reproducible computation	30
D	Applied decision system	15

SCORING GUIDE 4 = correct result with valid reasoning; 3 = substantially correct; 2 = partial structure; 1 = relevant attempt; 0 = no relevant evidence.

PART I**Exercises****A. Conceptual foundations****Exercise 1. Elements and singleton sets [4 points]**

Let $A = \{1, 2, \{3\}\}$. Evaluate each statement and justify the answer: $2 \in A$; $\{2\} \subseteq A$; $3 \in A$; $\{3\} \in A$; $\{\{3\}\} \subseteq A$.

Exercise 2. The universe matters [4 points]

A database contains registered adults in three districts. Explain why the complement of the set 'vaccinated people' cannot be interpreted until the universe U is declared. Give two plausible choices of U and explain how they change the result.

Exercise 3. Relation, partial function or total function [4 points]

Explain the distinctions among an arbitrary relation $R \subseteq A \times B$, a partial function from A to B , and a total function $f: A \rightarrow B$. State the two tests required for totality.

Exercise 4. Implication is not causation [4 points]

A rule states: 'If a household is eligible, then it receives a payment.' Explain what $P \rightarrow Q$ says formally and why observing a payment does not prove eligibility or establish a causal effect of eligibility.

Exercise 5. Operational definitions [4 points]

The label 'high risk' appears in a proposed automated decision rule. List the minimum controls required before this label can be translated into code.

B. Mathematical reasoning**Exercise 6. Set operations in a declared universe [7 points]**

Let $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $A = \{1, 2, 3, 5\}$, and $B = \{3, 4, 5, 7\}$.

- Compute $A \cup B$, $A \cap B$, $A \setminus B$, $B \setminus A$, A^c and B^c relative to U .
- Verify $(A \cup B)^c = A^c \cap B^c$ and $(A \cap B)^c = A^c \cup B^c$.

Exercise 7. Cartesian products [5 points]

Let $A = \{1, 2, 3\}$ and $B = \{a, b\}$. Enumerate $A \times B$ and $B \times A$. State their cardinalities and explain why the two products are not equal even though their cardinalities are equal.

Exercise 8. Truth tables and De Morgan's laws [7 points]

Construct one complete table containing P , Q , $\neg P$, $\neg Q$, $\neg(P \wedge Q)$, $(\neg P) \vee (\neg Q)$, $\neg(P \vee Q)$, and $(\neg P) \wedge (\neg Q)$. Use the rows to establish both logical equivalences.

Exercise 9. Valid and invalid inference [5 points]

For each argument, decide whether the conclusion follows and name the pattern: (a) $P \rightarrow Q$, P , therefore Q ; (b) $P \rightarrow Q$, Q , therefore P ; (c) $P \rightarrow Q$, $\neg Q$, therefore $\neg P$.

Exercise 10. Classifying finite mappings [6 points]

Let $A = \{1, 2, 3\}$, $B = \{a, b, c\}$, $R_1 = \{(1, a), (2, b), (3, c)\}$, $R_2 = \{(1, a), (1, b), (2, c)\}$, and $R_3 = \{(1, a), (2, a)\}$. Classify each as an arbitrary relation, partial function or total function from A to B . For every total function, determine whether it is injective and surjective.

Exercise 11. Relation matrix [5 points]

For patients $P = \{\text{Awa, Binta, Cheikh}\}$ and clinics $C = \{\text{North, South}\}$, let $R = \{(\text{Awa}, \text{North}), (\text{Binta}, \text{South}), (\text{Cheikh}, \text{North})\}$. Construct the 3×2 binary relation matrix using the stated row and column order. Recover the ordered pairs from the matrix.

C. Reproducible computation**Exercise 12. Symmetric difference from first principles [7 points]**

Write a Python function that returns elements belonging to exactly one of two finite sets without calling `set.symmetric_difference` or using the `^` operator. Test it on $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$.

Exercise 13. Validating a declared function [8 points]

Write a function `classify_relation(relation, domain, codomain)` that reports whether a finite relation is a function, whether it is total on the declared domain, and—when total—whether it is injective and surjective. Reject pairs containing values outside the declared domain or codomain.

Exercise 14. Matrix-to-relation conversion [6 points]

Write a function converting a binary matrix plus row and column labels into a set of ordered pairs. Validate dimensions, require entries to be 0 or 1, and reproduce the patient-clinic relation from Exercise 11.

Exercise 15. Predicate with explicit missing-data handling [9 points]

Implement the vaccination-priority rule: `age ≥ 65 OR health_worker OR vulnerable`, excluding `fully_vaccinated`. A record containing `None` in any required field must receive status `REVIEW` rather than silently treating the value as `False`. Return both status and a human-readable reason; test at least five records.

D. Applied decision system

Exercise 16. Transparent programme-eligibility engine [15 points]

Design a small eligibility engine for an emergency-assistance programme. A household qualifies when it is poor or food-insecure, has not received equivalent assistance, and belongs to an eligible district.

- Declare the universe U and operational definitions for P , F , B and G .
- Express the rule $E = (P \cup F) \cap B^c \cap G$ and define missing-data treatment.
- Implement the rule for at least eight synthetic households.
- Return ELIGIBLE, NOT ELIGIBLE or REVIEW plus reasons for every decision.
- Record a rule version, effective date, test cases and environment manifest.
- State two safeguards against unfair or untraceable decisions.

PART II

Complete solutions

INSTRUCTOR NOTE Equivalent notation and sound alternative implementations should receive full credit when the declared universe, domains, assumptions and validation evidence are explicit.

Solution 1. Elements and singleton sets

The statements are evaluated against the objects that actually occur in A.

- $2 \in A$: true.
- $\{2\} \subseteq A$: true, because the sole element 2 belongs to A.
- $3 \in A$: false; A contains the set $\{3\}$, not the number 3.
- $\{3\} \in A$: true.
- $\{\{3\}\} \subseteq A$: true, because its sole element $\{3\}$ belongs to A.

Solution 2. The universe matters

A complement is defined as $A^c = U \setminus A$, so changing U changes the people against whom non-membership is assessed.

- If U is all registered adults in the three districts, A^c contains registered adults without a vaccination record.
- If U is all residents of those districts, A^c may also contain children and unregistered residents.
- The second interpretation is larger and may mix 'not vaccinated' with 'not observed in the registry'.

Solution 3. Relation, partial function or total function

A relation is any subset of $A \times B$. It becomes functional when no input has more than one output. It is total from A when every declared input is mapped.

- Uniqueness test: each represented $a \in A$ is paired with at most one $b \in B$.
- Coverage test: each $a \in A$ is paired with at least one $b \in B$.
- Together, the tests imply exactly one output for every declared input.

Solution 4. Implication is not causation

$P \rightarrow Q$ is false only when P is true and Q is false. From $P \rightarrow Q$ and Q, P does not follow: Q may have another explanation. Formal implication also contains no intervention, temporal order, mechanism or control of confounding.

Solution 5. Operational definitions

Before coding 'high risk', approve and record:

- the measurable criteria and applicable population;
- data sources, provenance and effective date;
- missing-value and conflict-resolution rules;
- rule owner, version and review/appeal process;
- tests for expected, boundary and invalid cases.

Solution 6. Set operations in a declared universe

Direct calculation gives:

- $A \cup B = \{1, 2, 3, 4, 5, 7\}$; $A \cap B = \{3, 5\}$.
- $A \setminus B = \{1, 2\}$; $B \setminus A = \{4, 7\}$.
- $A^c = \{4, 6, 7, 8\}$; $B^c = \{1, 2, 6, 8\}$.
- $(A \cup B)^c = \{6, 8\} = A^c \cap B^c$.
- $(A \cap B)^c = \{1, 2, 4, 6, 7, 8\} = A^c \cup B^c$.

Solution 7. Cartesian products

$A \times B = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)\}$. $B \times A = \{(a, 1), (a, 2), (a, 3), (b, 1), (b, 2), (b, 3)\}$. Both have $3 \times 2 = 6$ elements, but ordered pairs reverse roles, so the sets are not equal.

Solution 8. Truth tables and De Morgan's laws

The matching column pairs establish both equivalences in every possible case.

P	Q	$\neg P$	$\neg Q$	$\neg(P \wedge Q)$	$\neg P \vee \neg Q$	$\neg(P \vee Q)$	$\neg P \wedge \neg Q$
T	T	F	F	F	F	F	F
T	F	F	T	T	T	F	F
F	T	T	F	T	T	F	F
F	F	T	T	T	T	T	T

Solution 9. Valid and invalid inference

The classifications are:

- (a) Valid: modus ponens.
- (b) Invalid: affirming the consequent.
- (c) Valid: modus tollens.

Solution 10. Classifying finite mappings

R_1 is a total function, injective and surjective. R_2 is not a function because input 1 has two outputs; it is therefore an arbitrary relation. R_3 is a partial function because input 3 is unmapped; it is not total from A to B. If considered only on represented inputs $\{1, 2\}$, it is non-injective.

Solution 11. Relation matrix

With rows (Awa, Binta, Cheikh) and columns (North, South), the matrix is $\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$. Reading every 1-entry recovers exactly $\{(Awa, North), (Binta, South), (Cheikh, North)\}$.

Solution 12. Symmetric difference from first principles

An element is retained exactly when its membership truth values differ.

```
def symmetric_difference_first_principles(a, b):
    return {x for x in a | b if (x in a) != (x in b)}

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
result = symmetric_difference_first_principles(A, B)
assert result == {1, 2, 5, 6}
```

Solution 13. Validating a declared function

Validate types first, then uniqueness, coverage and output use.

```
def classify_relation(relation, domain, codomain):
    if any(x not in domain or y not in codomain for x, y in relation):
        raise ValueError("Pair outside declared domain or codomain")
    mapping = {}
    functional = True
    for x, y in relation:
        if x in mapping and mapping[x] != y:
            functional = False
        mapping.setdefault(x, y)
    total = functional and set(mapping) == set(domain)
    injective = total and len(set(mapping.values())) == len(mapping)
    surjective = total and set(mapping.values()) == set(codomain)
    return {"function": functional, "total": total,
            "injective": injective, "surjective": surjective}

assert classify_relation({(1,"a"),(2,"b")},{1,2},{1,"a","b"}}["surjective"]
assert not classify_relation({(1,"a"),(1,"b")},{1},{1,"a","b"}}["function"]
```

Solution 14. Matrix-to-relation conversion

The implementation checks geometry and binary content before translating 1-entries.

```
def matrix_to_relation(matrix, row_labels, column_labels):
    if len(matrix) != len(row_labels):
        raise ValueError("Row count does not match labels")
    if any(len(row) != len(column_labels) for row in matrix):
        raise ValueError("Column count does not match labels")
    if any(value not in (0, 1) for row in matrix for value in row):
        raise ValueError("Matrix must be binary")
    return {(row_labels[i], column_labels[j])
            for i, row in enumerate(matrix)
            for j, value in enumerate(row) if value == 1}

M = [[1,0],[0,1],[1,0]]
expected = {("Awa", "North"), ("Binta", "South"), ("Cheikh", "North")}
assert matrix_to_relation(M, ["Awa", "Binta", "Cheikh"], ["North", "South"]) == expected
```

Solution 15. Predicate with explicit missing-data handling

A three-state decision prevents unknown values from being silently converted to rejection or acceptance.


```

REQUIRED = ("age", "health_worker", "vulnerable", "fully_vaccinated")

def priority_decision(person):
    missing = [key for key in REQUIRED if person.get(key) is None]
    if missing:
        return "REVIEW", "Missing: " + ", ".join(missing)
    priority = (person["age"] >= 65 or person["health_worker"]
               or person["vulnerable"])
    if person["fully_vaccinated"]:
        return "NOT ELIGIBLE", "Already fully vaccinated"
    if priority:
        reasons = []
        if person["age"] >= 65: reasons.append("age at least 65")
        if person["health_worker"]: reasons.append("health worker")
        if person["vulnerable"]: reasons.append("clinically vulnerable")
        return "ELIGIBLE", "; ".join(reasons)
    return "NOT ELIGIBLE", "No priority condition satisfied"

```

Minimum tests: an older unvaccinated person; a healthcare worker; a vulnerable but fully vaccinated person; a person satisfying no inclusion condition; and a record with one required value set to None. Assertions should verify both status and reason.

Solution 16. Transparent programme-eligibility engine

One acceptable design uses explicit set membership and sends incomplete records to REVIEW before evaluating the Boolean rule.

- U is the versioned set of households registered for the programme at the effective date.
- P, F, B and G must be generated from approved operational definitions and traceable source fields.
- For complete records, $\text{eligible}(h) = (h \in P \text{ or } h \in F) \text{ and } h \notin B \text{ and } h \in G$.
- Any missing required membership status produces REVIEW, not an assumed False.
- The output must store decision, reasons, rule version, effective date and source-data version.

```

RULE_VERSION = "EA-1.0"
REQUIRED = ("poor", "food_insecure", "prior_assistance", "eligible_district")

def decide(household):
    missing = [key for key in REQUIRED if household.get(key) is None]
    if missing:
        return "REVIEW", "Missing: " + ", ".join(missing)
    inclusion = household["poor"] or household["food_insecure"]
    if not inclusion:
        return "NOT ELIGIBLE", "No inclusion condition satisfied"
    if household["prior_assistance"]:
        return "NOT ELIGIBLE", "Equivalent assistance already received"
    if not household["eligible_district"]:
        return "NOT ELIGIBLE", "District is not eligible"
    return "ELIGIBLE", "Inclusion satisfied; no exclusion applies"

```

Safeguards include independent rule review, version-controlled definitions, boundary and regression tests, decision logs, periodic disparate-impact monitoring, and a documented human appeal mechanism.

Reproducibility checklist

- Run the companion notebook from a fresh Google Colab or isolated local environment.
- Record Python and package versions and use the declared seed where randomness is introduced.
- Preserve input data or synthetic fixtures, expected outputs and executable assertions.
- Declare universes, domains, codomains, missing-data rules and conflict precedence.
- Version policy rules and retain reasons for every acceptance, rejection or review decision.
- Confirm that all tests pass before updating checksums and the release record.

Alignment record

This companion is derived from PU-B01-C02_Chapter2_Audited_Controlled_Edition_v1.0 and M1_N02_sets_logic_relations_functions.ipynb. It expands the seven laboratory prompts and notebook exercises into a standalone assessed resource while preserving the chapter's definitions, examples, safeguards and SRAI Principle No. 2.

Controlled references

- OpenStax (2023). Contemporary Mathematics, §§1.1, 1.4-1.5.
- Python Software Foundation. Python 3 documentation, §5.4 Sets.
- PostgreSQL Global Development Group. PostgreSQL SELECT and table-expression documentation.
- W3C (2012). OWL 2 Web Ontology Language Primer, Second Edition.
- National Academies of Sciences, Engineering, and Medicine (2019). Reproducibility and Replicability in Science.